

CPSC 375 High-Performance Computing

Lecture 14

Thread Synchronization

Condition Variables

Exam I

- Friday, February 27
- Covers up to Lecture 14 (today)
- Format
 - Multiple-choice
 - Terminology and basic concepts
 - Short-answer questions
 - Homework exercises
 - Assignments

Why Condition Variables?

- Mutexes: "Who can access the data?"
 - Protect the integrity of shared memory
- Condition variables (CVs): "When is the data ready?"
 - Coordinate thread timing
- Without CVs, threads must *busy-wait*, wasting CPU cycles.
- CVs allow a thread to *yield* the processor voluntarily while waiting.
 - Minimize system overhead while maximizing thread activity.
 - Essential for any event-driven parallel architecture.

The Busy-Wait Problem

```
while (count == 0) ; // Spinning...
```

- Consumes 100% of a CPU core for zero productive work.
- Prevents other peer threads from utilizing that core's execution stream.
- Creates massive memory bus contention.
- Primary cause of poor *speedup* in parallel benchmarks
- CVs to transition a thread from *Running* to *Blocked* state.
 - OS manages this transition via the scheduler.
 - Moves from inefficient polling to efficient interrupt-driven logic.

What is a Condition Variable?

- A synchronization object specifically designed for signaling events.
 - Acts as a waiting room where threads pause until a condition changes.
- **CVs must always be used with a mutex.**
 - Mutex: necessary mutual exclusion for the shared data.
 - CV: condition synchronization to avoid race conditions.
- Data type: `pthread_cond_t`
 - Unlike semaphores, CVs do not have an internal counter (*stateless*).
 - If a signal is sent and no one is waiting, the signal is discarded (*memoryless*).

Components of Signaling

- Requirements
 - Mutex: protects the shared variable.
 - Condition variable: signaling channel for the wait/signal calls.
 - Predicate: boolean expression being checked (e.g., $\text{items} < 10$).
- Coordination requires all three to work in perfect synchronization.
 - Signaling without changing the predicate results in false wakeups.
 - Changing the predicate without signaling results in deadlock (*starvation*).

Initializing Condition Variables

- **Static Initialization:**

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

- **Dynamic Initialization: for heap-allocated CVs.**

```
pthread_cond_init(&cond, NULL);
```

- **Attributes:** `NULL` or can specify if the CV is process-private or process-shared.

- **Cleanup:** `pthread_cond_destroy()`

- Never destroy a CV while threads are actively waiting on it.

Waiting for a Condition

```
int pthread_cond_wait(pthread_cond_t *cond,  
                      pthread_mutex_t *mutex);
```

- **Three-step atomic sequence:**
 1. Releases the mutex so others can enter the critical section.
 2. Puts the thread to sleep.
 3. Re-acquires the mutex automatically upon being woken up.
- **Atomicity prevents the lost wakeup race condition.**
 - The thread must hold the mutex before calling this function.
 - It returns only after the thread has successfully re-acquired the lock.

Why Release the Mutex?

- If the waiter kept the mutex while sleeping, the system would freeze.
 - A producer thread would be locked out of the critical section.
 - Unable to add data or signal, leading to deadlock.
- The automatic release allows the producer to enter and change state.
- The automatic re-acquire ensures the consumer wakes up in a safe region.
- Without the release, no progress is possible once the buffer is full.

Signaling a Thread

```
int pthread_cond_signal(pthread_cond_t *cond);
```

- Wakes up at least one thread currently waiting on the CV.
- OS scheduler determines which thread in the queue is woken.
- Use this when only one thread can proceed.
- Signaling triggers the V (post) equivalent in semaphore logic.
- Signaling is memoryless; it is not stored for future threads.
- Make sure to hold the mutex while signaling to ensure atomicity of state.

Broadcasting to All

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

- Wakes up ALL threads currently blocked on the CV.
 - It is the global notification for thread groups and pools.
 - Essential for state changes that affect everyone
 - e.g., program shutdown
- Woken threads compete for the same mutex
 - All but one will re-block
- Use it only when multiple threads can truly make progress.
- Can lead to high system time due to context switching.
- Also used when multiple resources become available simultaneously.

The Predicate Loop

- Always use a `while` loop, never an `if` statement.

```
while (count == 0) // predicate loop
    pthread_cond_wait(&cond, &mutex);
```

- Why?
 - OS waked the thread for internal reasons (false wakeup).
 - Another thread took the item before you woke up.
- Re-checking the condition ensures the thread's state is logically sound.
- Protects the program from non-deterministic scheduler behavior.

The Producer-Consumer Model

- One producer thread and one consumer thread, and one shared *bounded buffer*
- Producer adds data; Consumer processes data.
- Shared data:

```
int buffer[10]           // bounded buffer
int count                // # items
int head                 // for consumer
int tail                 // for producer
```

- Synchronization:

```
pthread_mutex_t *lock    // mutex
pthread_cond_t *empty    // has an empty slot?
pthread_cond_t *full     // full of at least one item?
```

The Producer-Consumer: Initialization

```
int buffer[10];    // data conduit between threads
int head = 0;      // for the consumer's exit point
int tail = 0;      // for the producer's entry point
int count = 0;     // the number of elements in the buffer

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
                    // for index and count updates
pthread_cond_t empty = PTHREAD_COND_INITIALIZER;
                    // for the producer to wait for available space
pthread_cond_t full = PTHREAD_COND_INITIALIZER;
                    // for the consumer to wait for available data
```

The Producer Thread

```
while (1) {
    pthread_mutex_lock(&lock);

    // Wait while the buffer is full (no empty slots)
    while (count == 10)
        pthread_cond_wait(&empty, &lock);

    // Add item to buffer
    buffer[tail] = rand();
    printf("Producer: Inserted %d at index %d\n", i, tail);
    tail = (tail + 1) % 10;
    count++;

    // Signal that the buffer is now full of at least one item
    pthread_cond_signal(&full);
    pthread_mutex_unlock(&lock);
}
```

The Consumer Thread

```
while (1) {
    pthread_mutex_lock(&lock);

    // Wait while the buffer is empty (needs "full" slots)
    while (count == 0)
        pthread_cond_wait(&full, &lock);

    // Remove item from buffer
    int item = buffer[head];
    printf("Consumer: Extracted %d from index %d\n", item, head);
    head = (head + 1) % 10;
    count--;

    // Signal that an empty slot is now available
    pthread_cond_signal(&empty);
    pthread_mutex_unlock(&lock);
}
```


Kernel Transitions

- Uncontended Lock: Handled in user mode (fast)
- Contended Lock: Requires a context switch to kernel mode (slow)
- High lock contention shifts CPU time from user to system.
 - e.g., If 8 threads fight for 1 lock, performance will plummet.
- Process data outside the critical section whenever possible
- Keep the critical section as small as possible (fine-grained)

Buffer Size vs. Workload

- Buffer size: the physical capacity of the shared memory array.
- Workload: the total number of items to be moved through the system.
- If workload (20) > buffer size (10), the producer must yield to the consumer.
 - Locking once for 20 items in a 10-slot buffer = Instant Deadlock.
 - The producer will fill 10, sleep, and never release the lock for the consumer.
 - Fine-grained locking (20 separate locks) is required.
- The buffer size acts as a speed limit for the system.

Thread Stacks

- CVs and mutexes must live in shared memory" (global or heap).
- Each thread has its own private stack for local variables.
 - `pthread_cond_t` inside a thread function is invisible to others.
 - Results in signal loss and permanent thread hangs.
- Always ensure your sync objects have global visibility to all peer threads.

Timed Waits

- Sometimes you don't want to wait forever for a signal.

```
int pthread_cond_timedwait(  
    pthread_cond_t *cond,  
    pthread_mutex_t *mutex,  
    const struct timespec *abstime);
```

- This call allows a thread to specify a timeout period.
- Returns 0 if signaled, or ETIMEDOUT if the time expires.
- Uses absolute time (e.g., "Wait until 5:00 PM"), not relative time.
- Calculation requires functions such as `clock_gettime`.
- Useful for distributed jobs where a node might crash and never signal.

Building a Barrier

- A *barrier* ensures all threads reach a point before anyone proceeds.
- You can build one using 1 mutex, 1 CV, and a simple counter.
 - Threads increment the counter and wait if count $< N$.
 - The last thread sets the flag and calls broadcast.
- Used for phase-based parallel algorithms
 - e.g., matrix multiply
- Barriers synchronize the timeline of the entire parallel process.

Barrier Example

```
thread(tid) {  
    // Phase 1  
    do_this();  
    sync_barrier(tid);  
  
    // Phase 2  
    do_that();  
}
```

```
sync_barrier(tid) {  
    pthread_mutex_lock(&mtx);  
    int my_phase = phase;  
    count++;  
    if (count < THREAD_COUNT) {  
        while (my_phase == phase)  
            pthread_cond_wait(&cv, &mtx);  
    } else { // last thread  
        count = 0;  
        phase++;  
        pthread_cond_broadcast(&cv);  
    }  
    pthread_mutex_unlock(&mtx);  
}
```

